Universidade Federal de São Paulo
Instituto de Ciência e Tecnologia
Arquitetura e Organização de Computadores - Turma IB

Perceptual Hash para verificação de uso indevido de obras artísticas

Integrantes:
Breno Cerqueira Reis Nakamura - 168765
Estevan Teixeira Santos - 168826
Vinícius Augusto Landim Andrade - 169251
Vinícius Martins Goulart - 170502
Docente responsável: Profa. Dra. Thaína A. A. Tosta

São José dos Campos
2024/2

Universidade Federal de São Paulo
Instituto de Ciência e Tecnologia
Arquitetura e Organização de Computadores - Turma IB

Perceptual Hash para verificação de uso indevido de obras artísticas

Terceiro relatório do projeto de Arquitetura e Organização de Computadores (Turma IB) da Universidade Federal de São Paulo para cumprimento dos requisitos de avaliação e aprovação na disciplina.

São José dos Campos
2024/2

Conteúdo

1 Introdução **2**
1.1 Contextualização 2
1.2 Problema de Pesquisa 2

2 Objetivos **3**
2.1 Objetivo Geral 3
2.2 Objetivos Específicos 3

3 Implementação **3**
3.1 Localização das Imagens 4
3.2 Pré-processamento 4
3.2.1 Conversão para Escala de Cinza e Redimensionamento 4
3.3 Extração das Características 5
3.3.1 Transformada Discreta de Cosseno (DCT) 5
3.3.2 Implementação em Python 6
3.4 Geração do Hash Perceptual (pHash) 7
3.4.1 Leitura Arquivos .bin 7
3.4.2 Cálculo pHash 7
3.4.3 Mediana dos Coeficientes 8
3.4.4 Conversão para Hash Binário 8
3.4.5 Implementação do pHash 9
3.5 Métrica de Similaridade 10
3.5.1 Distância de Hamming 10
3.5.2 Normalização da Distância de Hamming 10

4 Resultados Obtidos **11**
4.1 Comparação entre Variações de uma Mesma Obra 11
4.2 Comparação entre Obras Distintas 13
4.3 Comparação entre Imagens Originais e Editadas 14
4.4 Comparação entre Obra Original e Imagem Gerada por IA . . 16

5 Conclusão **17**

6 Planejamento e cronograma de execução **18**

Resumo

Este trabalho apresenta o desenvolvimento e implementação de um sistema de verificação de autenticidade de imagens digitais baseado em funções de Perceptual Hash (pHash), visando auxiliar artistas na proteção de suas obras contra uso não autorizado, especialmente no contexto do avanço das tecnologias de inteligência artificial generativa. O sistema integra código em Python e Assembly MIPS, aproveitando as vantagens de cada linguagem para criar uma solução eficiente. A implementação se baseia em quatro etapas fundamentais: pré-processamento das imagens através de redimensionamento e conversão para escala de cinza, extração de características utilizando a Transformada Discreta de Cosseno (DCT), geração de hash perceptual de 64 bits e cálculo de similaridade através da Distância de Hamming Normalizada. Os testes realizados com obras artísticas conhecidas demonstraram a capacidade do sistema em identificar com precisão diferentes níveis de similaridade: entre variações de uma mesma obra (0.015 a 0.093), entre obras completamente distintas (0.437), em casos de edições e manipulações (0.062 a 0.156) e na comparação com imagens geradas por IA (0.468). Os resultados obtidos indicam que o sistema desenvolvido pode servir como uma ferramenta eficaz para artistas monitorarem e verificarem o uso de suas obras no ambiente digital, contribuindo para a proteção de direitos autorais em um cenário de crescente complexidade tecnológica.

1 Introdução

1.1 Contextualização

A disseminação de imagens na internet trouxe desafios para os direitos autorais, agravados pelo avanço da Inteligência Artificial (IA). Ferramentas de IA generativa, treinadas com grandes volumes de dados extraídos da web, criam novas imagens sem transparência sobre suas fontes, dificultando o consentimento dos artistas (JIANG et al., 2023). Isso afeta o mercado artístico, tornando a concorrência desigual e comprometendo a propriedade intelectual. A ilustradora Lúcia Lemos relata: "A gente já está perdendo emprego, tendo nosso trabalho roubado para treinar essas ferramentas"(TAVARES, 2024).

Outrossim, dado o avanço da IA, modelos de aprendizado profundo ajustam automaticamente parâmetros como exposição e contraste, além de selecionar e filtrar imagens, com base na composição e nas expressões faciais, também sendo capazes de efetuar a criação de imagens com a fusão de múltiplas imagens (CHEN, 2024). A IA também pode corrigir defeitos e reduzir ruídos para melhorar a qualidade e o impacto visual das imagens. Embora essas ferramentas ampliem as possibilidades criativas, elas também impõem desafios para a autenticidade e verificação da propriedade intelectual.

Nesse cenário, funções de *Perceptual Hash* (pHash) surgem como solução para autenticar e proteger imagens, tornando o pHash uma ferramenta essencial para detectar alterações e preservar a integridade das imagens. O pHash extrai características únicas e gera um hash que permite detectar modificações não autorizadas (DU; HO; CONG, 2020).

1.2 Problema de Pesquisa

Diante dos desafios impostos pela IA na criação e edição de imagens, garantir a autenticidade e a autoria de obras visuais tornou-se um problema crítico. Métodos tradicionais de proteção por meio de metadados e marcas d'água são insuficientes diante da manipulação automatizada e da disseminação em larga escala na internet. Nesse contexto, este trabalho investiga o uso do Perceptual Hash (pHash) como ferramenta para verificação de autenticidade de imagens, explorando sua capacidade de detectar edições, transformações sutis e imagens semelhantes. A pesquisa busca avaliar a eficácia do pHash em diferentes cenários de manipulação digital, discutindo suas limitações e potencial como um método de preservação da integridade das imagens na era da inteligência artificial.

2 Objetivos

2.1 Objetivo Geral

Desenvolver uma forma de verificar a autenticidade de imagens digitais utilizando *Perceptual Hash* (pHash), a fim de auxiliar artistas na identificação de possíveis usos indevidos de suas obras por Inteligências Artificiais generativas.

2.2 Objetivos Específicos

- Implementar um algoritmo de *Perceptual Hash* baseado na *Transformada Discreta de Cosseno* (DCT), combinando Python e Assembly MIPS para um processamento eficiente.
- Criar uma métrica de similaridade baseada na *Distância de Hamming*, permitindo a detecção do grau de semelhança entre imagens.
- Comparar pHashs gerados de diferentes imagens para identificar cópias idênticas e variações sutis que possam indicar uso indevido.
- Avaliar a robustez do algoritmo em diferentes cenários de manipulação digital, como mudanças de cor, redimensionamento e sobreposição de elementos gráficos.
- Integrar o código Python e Assembly para permitir um fluxo de processamento contínuo e eficiente.
- Expandir o hash gerado para 64 bits, garantindo maior precisão na verificação de similaridade.

3 Implementação

No que tange à implementação do projeto, utilizaram-se as linguagens de programação Python e Assembly na arquitetura MIPS de 32 bits. O pHash a ser desenvolvido é um *DCT Based Hash*, ou seja, que faz uso da Transformada Discreta de Cosseno (DCT), conforme será detalhado adiante. Esse algoritmo é estruturado em 4 etapas: Pré-processamento, extração das características, quantização, geração do hash e obtenção de métricas de similaridade (SAMANTA; JAIN, 2021, p.205).

3.1 Localização das Imagens

Antes da etapa de pré-processamento, um pequeno algoritmo em Assembly MIPS será responsável por capturar a localização das imagens no computador(diretório) até as duas imagens que serão comparadas. O usuário fornecerá um "caminho"para as duas imagens, separando-as por espaço em branco. Esse input será armazenado em um buffer, cujo conteúdo será escrito em arquivo de texto. Através do comando "li $v0, 13", o arquivo .txt será criado e preparado para a escrita ou sobrescrito caso já exista. Posteriormente, o comando chave "li $v0, 15" habilita a escrita com o auxílio dos registradores $a0(contém um número gerado por $v0 na etapa anterior, que serve como identificador do arquivo que sofrerá a operação) e $a1(apontará para o conteúdo do buffer). Caso nenhum erro de escrita seja identificado ($v0 retorna um valor menor que 0), o arquivo é fechado e a escrita é completa.

3.2 Pré-processamento

Partindo da etapa de pré-processamento, a imagem de entrada que será avaliada será preparada para a extração das informações, reduzindo o tamanho dos dados que serão processados.

3.2.1 Conversão para Escala de Cinza e Redimensionamento

Inicialmente a imagem é convertida em uma escala de cinza, de modo a extrair informações baseadas apenas na luminância, descartando as variações de cor que não influenciam significativamente na percepção visual do conteúdo. Com isso feito, a imagem é redimensionada para um tamanho reduzido de 32x32, diminuindo a quantidade de dados e preservando as informações relevantes (ZAUNER, 2010, p.30)

Esse processo foi realizado utilizando Python com a biblioteca *Pillow*, executando-se os seguintes comandos::

```
from PIL import Image

image = Image.open(image_path).convert("L")
resized_image = image.resize((dct_matrix_size * 4,
    dct_matrix_size * 4), Image.LANCZOS)
```

Código 1: Trecho de código para pré-processamento da imagem

Na operação de *resize*, conforme mostrada pelo Código 1 temos a multiplicação de `dct_matrix_size`, que é definida pelo tamanho da matriz DCT desejada – conforme será mostrado posteriormente – multiplicada pelo fator

4, que promove um redimensionamento em tamanhos padrões, como 16x16 e 32x32.

3.3 Extração das Características

A etapa de extração das características, é fundamental que as características extraídas da imagem permaneçam inalteradas mesmo diante de manipulações que não alterem o conteúdo da imagem. Ou seja, pequenas alterações na imagem, como variações na iluminação, escala ou rotação, não devem alterar significativamente as características extraídas (SAMANTA; JAIN, 2021, p.205).

3.3.1 Transformada Discreta de Cosseno (DCT)

Para essa finalidade, a Transformada Discreta de Cosseno (DCT), especificamente a variante de tipo II, é aplicada, sendo representada na Equação 1.

$$X[n] = \frac{\sqrt{2}}{N} \sum_{m=0}^{N-1} x[m] \cdot \cos\left(\frac{(2m+1)n\pi}{2N}\right), \quad (n = 0, \ldots, N-1) \tag{1}$$

onde $x[m]$, com $m = 0, \ldots, N-1$, é um vetor de entrada que representa uma sequência de N pontos.

Sendo a matriz DCT, definida pela Equação 2, a utilizada para obter-se as informações que serão computadas (ZAUNER, 2010, p.21-22).

$$c[n,m] = \frac{2}{\sqrt{N}} \cdot \cos\left(\frac{(2m+1) \cdot n\pi}{2N}\right), \quad (m, n = 0, \ldots, N-1) \tag{2}$$

Onde $c[n,m]$ é o coeficiente correspondente na posição (n,m) da matriz DCT.

Na prática, a DCT decompõe uma imagem em componentes de frequência, retornando uma matriz onde cada elemento representa a contribuição de uma frequência específica na imagem como um todo. No caso do *Perceptual Hash*, normalmente selecionam-se blocos comuns de 8x8 pixels, sendo os coeficientes no canto superior esquerdo correspondentes às frequências mais baixas, que carregam a maior parte da informação visual perceptível e são mais estáveis sob manipulações como compressão e redimensionamento. À medida que se move para os índices mais altos na matriz, os coeficientes representam frequências mais altas. Essas propriedades tornam os coeficientes de baixa frequência ideais para criar representações compactas e robustas da imagem, como para a aplicação do *Perceptual Hash* (ZAUNER, 2010, p.22 e 32).

3.3.2 Implementação em Python

Essa etapa também foi contemplada pelo programa em Python. Nele, a biblioteca *Numpy* foi utilizada para extrair os valores dos pixels da imagem de entrada, de modo a serem utilizados na aplicação da DCT. A transformação feita pela DCT é feita primeiramente por linhas e depois por colunas (ZAUNER, 2010, p.22). Para isso, usando a biblioteca *Scipy.fftpack*, a função `dct()` foi utilizada para obter a matriz com os valores dos coeficientes da transformada.

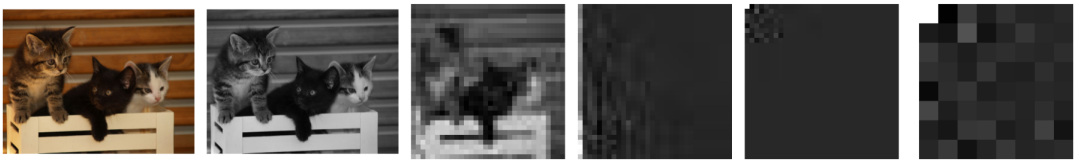
```
pixel_matrix = np.array(resized_image, dtype=np.float32)
dct_matrix = dct(dct(pixel_matrix, axis=0, type=2, norm='
    ortho'), axis=1, type=2, norm='ortho')
```

Código 2: Conversão de matriz e aplicação da Transformada Discreta de Cosseno

Assim, foram selecionados 8x8 coeficientes, obtendo-se um total de 64 coeficientes.

O procedimento executado nas etapas 1 e 2 é descrito visualmente por Ramos (2022) na Figura 1.

Figura 1: Etapas de pré-processamento e extração das características da imagem.



Fonte: Ramos, 2022

Por fim, a matriz será apresentada na forma de um vetor de tamanho 64 com a união das linhas da matriz, para se adequar ao formato de entrada para o cálculo do *pHash*.

Com as funções responsáveis por processar as imagens finalmente finalizadas, utilizando a função nativa do Python "with open(nome ou caminho até o arquivo, modo)" é possível acessar o arquivo .txt gerado pelo assembly, ativar o modo de leitura e salvar seu conteúdo em uma variável. Posteriormente, podemos quebrar a string obtida por espaços em branco com ".split", obter ambos caminhos digitados e chamar as funções desenvolvidas para obter o vetor final.

Para finalizar a implementação em python, utilizando a mesma função da leitura, utilizaremos o modo de escrita binária(operação = wp) para gerar dois arquivos .bin de modo que o Assembly seja capaz de acessar ambos vetores gerados para cada imagem. Utilizando o método ".write", selecionamos as variáveis em que os vetores estão contidos e passamos como argumento 64f para uma escrita em 64 bits.

3.4 Geração do Hash Perceptual (pHash)

Com todos os dados de entrada da imagem extraídos, a terceira etapa pode ser executada e o *pHash* pode ser finalmente calculado. A partir daqui, a implementação é feita em MIPS Assembly.

3.4.1 Leitura Arquivos .bin

O procedimento se inicia na leitura e armazenamento dos valores contidos no arquivo .bin gerado. Dois buffers são reservados para o vetor de cada imagem, ambos com o mesmo conteúdo, um de nome "dct_array" e outro de nome "dct_array_sorted", que será ordenado futuramente. As funções responsáveis pela leitura e armazenamento do conteúdo são completamente análogas, sendo sua única diferença os buffers em que os conteúdos serão armazenados e o arquivo a ser lido. De forma a seguir o seguinte formato: "read_file"(read_file1 para o vetor da imagem 1 e read_file2 para o vetor da imagem 2) armazenam os valores em dct_array(1 e 2, respectivamente). As funções read_fileSort1 e read_fileSort2 agem de forma análoga.

As funções começam na abertura do arquivo a ser lido e transferido, onde o processo de abertura é idêntico ao descrito no processo de geração do arquivo de texto. O código "li $v0, 14" é utilizado para iniciar o modo de leitura, onde o $a0 novamente referencia o arquivo a ser lido, $a1 aponta para o buffer onde o conteúdo será armazenado e $a2 para a quantidade de bits a serem lidos. Caso nenhum erro ocorra no procedimento, o código syscall para fechar o arquivo é chamado(de forma idêntica ao processo de escrita no arquivo .txt) e o processo de leitura e armazenamento se encerra.

3.4.2 Cálculo pHash

O algoritmo para cálculo do *pHash* se inicia com o cálculo do valor mediano do vetor dos coeficientes obtidos pela DCT. Uma vez que o valor mediano é encontrado, ele é comparado elemento a elemento de modo a formar um número binário.

3.4.3 Mediana dos Coeficientes

A implementação em Assembly inicia-se com o cálculo da mediana do vetor de coeficientes. Para isso, utiliza-se o algoritmo de *Bubble Sort* para ordenar o vetor `dct_array_sorted` com os coeficientes DCT. Com o vetor ordenado, a mediana é determinada calculando a média dos dois elementos centrais, já que neste caso há um vetor de 64 elementos.

3.4.4 Conversão para Hash Binário

O procedimento para geração do número binário é descrito da seguinte forma:

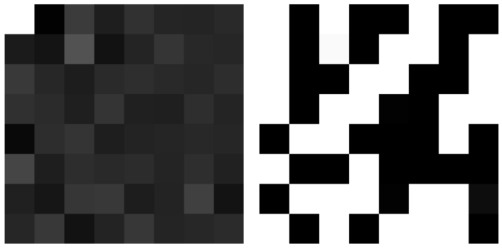
Seja m o valor mediano e os coeficientes do vetor definidos por C_i, em que $i = 0, \ldots, 15$, temos que:

$$h_i = \begin{cases} 0, & \text{se } C_i < m \\ 1, & \text{se } C_i \geq m \end{cases} \tag{3}$$

em que h_i é o bit do *pHash* na posição i. (ZAUNER, 2010, p.30)

Desse modo, o valor binário obtido será o *pHash*. A representação visual final do procedimento é descrita pela Figura 2.

Figura 2: Funcionamento visual da etapa de cálculo do *pHash*.



Fonte: Ramos, 2022

Após a obtenção da mediana, a operação definida pela Equação 3 é executada. Esse processo baseia-se na comparação de cada elemento do vetor original com o valor da mediana, gerando um número binário de saída. Para isso, foi desenvolvida a função `compare_element_with_median_loop`, que recebe, no registrador `$a1`, o endereço do vetor de coeficientes da figura correspondente.

3.4.5 Implementação do pHash

O algoritmo percorre o vetor de coeficientes do último ao primeiro elemento, respeitando a ordem dos bits gerados, indo do menos significativo (LSB) ao mais significativo (MSB).

Inicialmente, os registradores `$a2` e `$a3` são inicializados com o valor 0 para armazenar o número binário resultante. O registrador `$a2` guarda os 32 bits menos significativos, enquanto `$a3` armazena os 32 bits mais significativos. Essa separação ocorre devido à arquitetura de 32 bits do MIPS Assembly utilizada no projeto, que não permite um número formado por 64 bits. Assim, o algoritmo começa preenchendo `$a2` e, após processar os primeiros 32 elementos do vetor, continua percorrendo os restantes, armazenando-os em `$a3`. Durante o percurso, cada elemento do vetor é comparado com a mediana. Se o valor do elemento for maior ou igual a mediana, o registrador `$t4` recebe o valor imediato 1; caso contrário, apenas os índices são incrementados, como será detalhado posteriormente.

Quando `$t4` recebe o valor 1, a operação de deslocamento à esquerda *shift left* é aplicada para posicionar o bit na sua posição correspondente. Assim, como o algoritmo processa os coeficientes do vetor partindo do LSB, o primeiro deslocamento será à esquerda no total de zero posições, mantendo o bit 1 na sua posição inicial. O próximo bit será deslocado em uma posição, ficando na posição 2^1, e assim sucessivamente, até que o último bit seja deslocado 63 posições, alcançando a posição de MSB.

Após cada deslocamento, o registrador responsável pelo armazenamento do número binário é atualizado por meio de uma operação lógica OR entre ele e `$t4`, garantindo que o bit recém-posicionado seja incorporado ao resultado. Esse processo se repete para todos os elementos do vetor. Ao final da execução, os registradores `$a2` e `$a3` contêm o *pHash* calculado, um número binário de 64 bits correspondente ao tamanho do vetor de coeficientes DCT.

Note que, no caso do valor do elemento ser menor que mediana, o algoritmo apenas avança para o próximo índice sem modificar o número binário. Isso ocorre porque, ao inicializar os registradores `$a2` e `$a3` com zero, o número binário de saída já começa com todos os bits em 0. Assim, quando a operação OR é aplicada para posicionar um bit 1, os bits que permaneceram inalterados continuam sendo 0, sem necessidade de manipulação adicional.

Portanto, o resultado binário é exibido em seu formato binário. No entanto, como o MIPS Assembly não possui uma chamada direta para isso, a função `print_binary` foi desenvolvida para impressão. A função recebe, como parâmetro, o registrador `$a1`, que deve conter o endereço do registrador cujo número binário será impresso.

O processo consiste em deslocar o bit desejado para a posição menos

significativa por meio de um deslocamento à direita *shift right*. Em seguida, uma operação lógica AND entre o número deslocado e o bit 1 é realizada para extrair o bit correspondente. Esse bit é então convertido para seu equivalente ASCII ('0' ou '1') e impresso no programa.

Exemplificando o procedimento, o algoritmo começa deslocando o número 31 posições à direita (equivalente ao tamanho do número binário no registrador menos um), posicionando o primeiro bit na posição menos significativa. Em seguida, aplica-se a operação AND com 1 para isolá-lo e convertê-lo para ASCII. Na iteração seguinte, o número é deslocado 30 posições, trazendo o segundo bit para a posição LSB, onde será extraído e impresso. Esse processo se repete, reduzindo o deslocamento em uma posição a cada iteração, até que todos os bits tenham sido percorridos e exibidos na sequência correta.

3.5 Métrica de Similaridade

Por fim, na etapa de obtenção de métricas de similaridade, optou-se pela utilização do algoritmo da Distância de Hamming, assim como descrito por Zauner (2010), uma das abordagens mais amplamente utilizadas.

3.5.1 Distância de Hamming

Esse método mede a quantidade de bits diferentes entre dois hashes, utilizando operações XOR para identificar as diferenças bit a bit entre os binários fornecidos, gerando um número em que cada bit 1 indica uma diferença, que é posteriormente contada em um loop, refletindo o grau de similaridade entre as imagens. Por conta da limitação de bits do MIPS assembly, que levou a divisão dos hashes binários de 64 bits em dois registradores de 32 bits cada, a comparação é feita considerando, primeiramente, os registradores de cada hash que compõe os primeiros 32 bits das sequências e, posteriormente, os registradores que possuem os 32 últimos bits de cada hash.

3.5.2 Normalização da Distância de Hamming

Na implementação utilizada, foi aplicada a variante normalizada da Distância de Hamming, na qual divide-se o resultado da quantidade de bits diferentes pelo comprimento das strings de análise — no caso, 64 bits. O resultado retorna um valor entre 0 (hashes idênticos) e 1, no qual valores próximos a 0 expressam semelhança entre as imagens e, como exposto por McKeown e Buchanan (MCKEOWN; BUCHANAN, 2023), para imagens não relacionadas, deve ter valores próximos de 0.5, por conta da aleatoriedade. Assim, permitindo a definição de um limiar para a classificação de imagens similares.

Para tal, foi necessária a conversão do valor para ponto flutuante, para que a divisão por 64 fosse corretamente retornada.

4 Resultados Obtidos

O projeto consiste em três implementações distintas, mas que se completam. A primeira é um programa em Assembly que permite a entrada dos caminhos das imagens a serem comparadas, os quais devem ser digitados sem aspas e separados por espaço. Após a compilação desse código, um segundo programa, agora em Python, processa as imagens especificadas, gerando os dados necessários para a análise. Na última etapa, um segundo código em Assembly utiliza esses dados para calcular os pHashes das imagens e realizar a comparação, extraindo as métricas de similaridade. Para avaliar o algoritmo, essa sequência de passo foi efetuada para cada teste.

4.1 Comparação entre Variações de uma Mesma Obra

Primeiramente, 2 variações de imagens da obra "Monalisa" de Leonardo da Vinci foram comparadas com uma versão definida como referência para o teste. Essas são:

Figura 3: Imagem Referência da Obra "Monalisa".

Fonte: WIKIPEDIA, 2010

Figura 4: Imagem 1 da Obra "Monalisa".

Fonte: WIKIPEDIA, 2010

Figura 5: Imagem 2 da Obra "Monalisa".

Fonte: WIKIPEDIA, 2010

O primeiro teste foi feito entre a Figura 3 e a Figura 4. Nele, o algoritmo calculou um valor de 0.015625 para a Distância de Hamming Normalizada. Desse modo, o algoritmo foi capaz de identificar que as imagens, apesar de suas divergências quanto a cores e saturação, são altamente semelhantes. Neste caso, obtemos um teste positivo, já que sabemos que ambas as figuras partem de mesma origem.

Figura 6: Resultados do Teste entre a Figura 3 e a Figura 4.

```
Hash (em binario) da Imagem 1: 1100110010110100101001111111100100001100101001011001100110010110 0
Hash (em binario) da Imagem 2: 1100110010110100101001111111100100001100101001011001100110010111 0

Distancia de Hamming: 1
Distancia de Hamming Normalizada: 0.015625
```

Fonte: Os Autores.

Comparando as Figuras 3 e 5, obtemos os seguintes resultados:

Figura 7: Resultados do Teste entre a Figura 3 e a Figura 5.

```
Hash (em binario) da Imagem 1: 1100110010110100101001111111001000011001010010110011001101001100
Hash (em binario) da Imagem 2: 1100110010101100101001111111001010011000110010110011001101001110

Distancia de Hamming: 6
Distancia de Hamming Normalizada: 0.09375
```

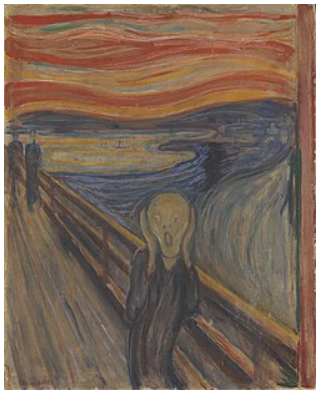
Fonte: Os Autores.

Note que, agora, além das diferenças entre cores e saturação, há um pequeno redimensionamento da imagem na Figura 7. Ainda assim, o resultado apontou semelhança, já que a Distância de Hamming Normalizada foi de 0.09375, sendo um valor bem próximo a zero.

4.2 Comparação entre Obras Distintas

A obra "Monalisa" foi comparada com a pintura "O Grito" de Edvard Munch, apresentada na Figura 8.

Figura 8: Imagem da Obra "O Grito".



Fonte: WIKIPEDIA, 2025

Para tal, as métricas obtidas passaram a indicar divergência, contabilizando um total de 28 dígitos do pHash binário que diferem-se, com um valor normalizado de 0.4375.

Figura 9: Resultados do Teste entre a Figura 3 e a Figura 8.

```
Hash (em binario) da Imagem 1: 1100110010110100101001111111100100001100101001011001100110100110 0
Hash (em binario) da Imagem 2: 1110010110001000111100111011110010001011011000010110111001011111

Distancia de Hamming: 28
Distancia de Hamming Normalizada: 0.4375
```

Fonte: Os Autores.

4.3 Comparação entre Imagens Originais e Editadas

Também, foram avaliadas edições feitas a partir de uma imagem referência. Para tal, a obra "Noite Estrelada" de Vicent van Gogh foi utilizada. Com a Figura 10 apresentando a imagem original e a Figura 11 suas variações; uma com a aplicação de um padrão de texto sob a figura e outra com uma edição de imagem.

Figura 10: Imagem da Obra "Noite Estrelada".

Fonte: WIKIPEDIA, 2025

Figura 11: Imagens da Obra "Noite Estrelada" alteradas.

Fonte: Os Autores.

Para o caso da comparação com a edição de texto, a Distância de Hamming Normalizada foi de 0.0625, identificando a autenticidade entre as imagens. No caso da edição da figura, foi obtido um total de 0.15625, valor no qual se encontra ainda relativamente próximo de zero, ou seja, ainda pode-se concluir pelo algoritmo que as figuras devão ser semelhantes.

Figura 12: Resultados do Teste entre a Figura 10 e a Figura 11 com edição de texto.

```
Hash (em binario) da Imagem 1: 1010111110100011001000000101110011010110001010000101101011010111
Hash (em binario) da Imagem 2: 1010111110100011001000000100111011010110001010000101100001010111

Distancia de Hamming: 4
Distancia de Hamming Normalizada: 0.0625
```

Fonte: Os Autores.

Figura 13: Resultado dos Testes entre a Figura 10 e a Figura 11 com edição de imagem.

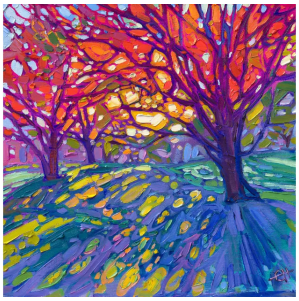
```
Hash (em binario) da Imagem 1: 1010111110100011001000000101110011010110001010000101101011010111
Hash (em binario) da Imagem 2: 1010111110100011000100000001110001000110001110000111111001000111

Distancia de Hamming: 10
Distancia de Hamming Normalizada: 0.15625
```

Fonte: Os Autores.

4.4 Comparação entre Obra Original e Imagem Gerada por IA

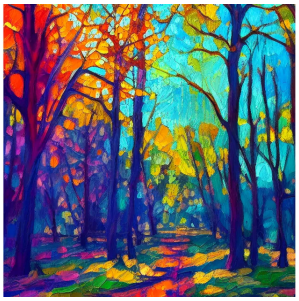
Por fim, o último dos testes foi feito baseando-se na matéria feita por Metz (2022) , que informa sobre o uso de obras de arte no treinamento de IAs. Metz (2022) gerou uma imagem usando a inteligência artificial *Stable Diffusion* com um comando descrevendo uma das obras da artista Erin Hanson. O resultado gerado pela tecnologia possuía, segundo a artista, características semelhantes às suas obras. A Figura 14 retrata a obra original da pintora e a Figura 15 apresenta a criada pela IA.

Figura 14: Imagem da Obra "*Crystalline Maples* "de Erin Hanson.



Fonte: METZ, 2024

Figura 15: Imagem da Imagem gerada pela Inteligência Artificial.



Fonte: METZ, 2024

Comparando ambas as imagens no algoritmo, o resultado foi de 0.46875 para a Distância de Hamming Normalizada, de modo a não ser capaz de alegar uma semelhança entre a imagem gerada artificialmente com a pintura original.

Figura 16: Resultados do Teste entre a Figura 14 e a Figura 15 com edição de texto.

```
Hash (em binario) da Imagem 1: 1110110101100011010000110011001110011001011000011100110001100011
Hash (em binario) da Imagem 2: 1011100110011110110001100110011110000110011001010100111010010110

Distancia de Hamming: 30
Distancia de Hamming Normalizada: 0.46875
```

Fonte: Os Autores.

5 Conclusão

O projeto desenvolveu um sistema de verificação de autenticidade de imagens utilizando funções de Perceptual Hash, combinando implementações em Python e Assembly MIPS, com o objetivo de auxiliar artistas na identificação de possíveis usos indevidos de suas obras. O sistema implementado é capaz de processar imagens digitais através de um processo que inclui pré-processamento, extração de características usando a Transformada Discreta de Cosseno (DCT), geração de hash perceptual de 64 bits e cálculo de similaridade através da Distância de Hamming Normalizada.

Os testes realizados demonstraram a eficácia do sistema em diferentes cenários. Na comparação entre variações de uma mesma obra (Monalisa), o algoritmo obteve valores de similaridade próximos a zero (0.015625 e 0.09375), indicando corretamente a semelhança entre as imagens, mesmo com alterações de cor, saturação e dimensionamento. Ao comparar obras distintas, como "Monalisa"e "O Grito", o sistema apresentou uma distância normalizada de 0.4375, demonstrando sua capacidade de diferenciar obras não relacionadas. Na análise de imagens editadas da obra "Noite Estrelada", o algoritmo manteve sua eficácia, identificando similaridade mesmo com a adição de texto (0.0625) e alterações na imagem (0.15625). Notavelmente, ao comparar uma obra original com sua versão gerada por IA, o sistema registrou uma distância de 0.46875, indicando baixa similaridade e demonstrando potencial para identificar conteúdo gerado artificialmente.

Entre as limitações do sistema, destaca-se a necessidade de ajuste fino dos limiares de similaridade para diferentes contextos e tipos de imagens. Como trabalhos futuros, sugere-se a implementação de técnicas adicionais de processamento de imagem para maior robustez contra transformações geométricas mais complexas, a otimização do desempenho computacional do código Assembly, o desenvolvimento de uma interface gráfica para facilitar a utilização por usuários não técnicos e a expansão do sistema para processar

múltiplas imagens simultaneamente.

6 Planejamento e cronograma de execução

Atividade	**Responsável**	**Meses**				
		Nov.	**Dez.**	**Jan/1**	**Jan/2**	**Fev.**
1. Levantamento bibliográfico (Entrega 1)	Breno Nakamura Estevan Teixeira Vinícius Andrade Vinícius Goulart					
2. Elaboração do relatório (Entrega 1)	Breno Nakamura Estevan Teixeira Vinícius Andrade Vinícius Goulart					
3. Levantamento bibliográfico (Entrega 2)	Vinícius Andrade Vinícius Goulart					
4. Implementação em Python (Entrega 2)	Estevan Teixeira					
5. Implementação em Assembly (Entrega 2)	Breno Nakamura Vinícius Andrade					
6. Implementar funcionalidade de comparação de semelhança entre duas imagens de entrada (Entrega Final)	Breno Nakamura					
7. Integração do código em Python com o Assembly (Entrega Final)	Estevan Teixeira Vinícius Goulart					
8. Expandir hash para 64 bits (Entrega Final)	Vinícius Andrade					
9. Elaboração do texto final (Entrega Final)	Breno Nakamura Estevan Teixeira Vinícius Andrade Vinícius Goulart					
10. Revisão do texto (Entrega Final)	Estevan Teixeira					

Referências

CHEN, Y. Artificial intelligence technology in photography and future challenges and reflections. *The Frontiers of Society, Science and Technology*, v. 6, p. 24–30, 2024. ISSN 2616-7433. Disponível em: ⟨https://francis-press.com/papers/16167\#location⟩.

DU, L.; HO, A. T.; CONG, R. Perceptual hashing for image authentication: A survey. *Signal Processing: Image Communication*, v. 81, p. 115713, 2020. ISSN 0923-5965. Disponível em: ⟨https://www.sciencedirect.com/science/article/pii/S0923596519301286⟩.

JIANG, H. H. et al. Ai art and its impact on artists. In: *Proceedings of the 2023 AAAI/ACM Conference on AI, Ethics, and Society*. New York, NY, USA: Association for Computing Machinery, 2023. (AIES '23), p. 363–374. ISBN 9798400702310. Disponível em: ⟨https://doi.org/10.1145/3600211.3604681⟩.

MCKEOWN, S.; BUCHANAN, W. J. Hamming distributions of popular perceptual hashing techniques. *Forensic Science International: Digital Investigation*, v. 44, p. 8, 2023. Disponível em: ⟨https://www.sciencedirect.com/science/article/pii/S2666281723000100?ref=pdf_download&fr=RR-2&rr=90f58821edd27dfb⟩.

METZ, R. *These artists found out their work was used to train AI. Now they're furious*. Estados Unidos: CNN Business, 2022. Disponível em: https://edition.cnn.com/2022/10/21/tech/artists-ai-images/index.html. Acesso em: 23 nov. 2024.

RAMOS, A. *Introduction to Perceptual Hashes: Measuring Similarity*. Espanha: Apiumhub, 2022. Disponível em: https://apiumhub.com/tech-blog-barcelona/introduction-perceptual-hashes-measuring-similarity/. Acesso em: 15 jan. 2024.

SAMANTA, P.; JAIN, S. Analysis of perceptual hashing algorithms in image manipulation detection. *Procedia Computer Science*, v. 185, p. 203–212, 2021. ISSN 1877-0509. Big Data, IoT, and AI for a Smarter Future. Disponível em: ⟨https://www.sciencedirect.com/science/article/pii/S1877050921011030⟩.

TAVARES, V. *Os ilustradores que já têm empregos e criações 'roubados' por IA: 'Medo de não conseguir me sustentar'*. São Paulo/SP: BBC News Brasil, 2024. Disponível em: https://www.bbc.com/portuguese/articles/c77lnezy1l3o. Acesso em: 23 nov. 2024.

WIKIPÉDIA. *Mona Lisa.jpg — Wikipédia, a enciclopédia livre.* 2010. [Online; accessed 3-fevereiro-2025]. Disponível em: ⟨\url{https://en.wikipedia.org/wiki/File:Mona_Lisa.jp}.⟩

WIKIPÉDIA. *A Noite Estrelada — Wikipédia, a enciclopédia livre.* 2025. [Online; accessed 3-fevereiro-2025]. Disponível em: ⟨\url{https://pt.wikipedia.org/w/index.php?title=A_Noite_Estrelada&oldid=6949117}.⟩

WIKIPÉDIA. *O Grito — Wikipédia, a enciclopédia livre.* 2025. [Online; accessed 5-fevereiro-2025]. Disponível em: ⟨\url{https://pt.wikipedia.org/w/index.php?title=O_Grito&oldid=6950695}.⟩

ZAUNER, C. *Implementation and Benchmarking of Perceptual Image Hash Functions.* Áustria: Upper Austria University of Applied Sciences, 2010. Disponível em: https://www.phash.org/docs/pubs/thesis$_{z}auner.pdf.Acessoem$: $15jan.$2024.